Dividir para ordenar

Intercalação e ordenação rápida

Encontro 12 | 15 de outubro

Estruturas de Dados Avançadas

De onde a gente parou

Semana passada: os métodos simples são $O(n^2)$,
e nenhum método por comparação passa de $n \log n$.

De onde a gente parou

Semana passada: os métodos simples são $O(n^2)$,
e nenhum método por comparação passa de $n \log n$.

Hoje: dois métodos que alcançam esse limite.

1
A primeira ideia

Dividir o trabalho

Você tem 100 provas para corrigir e um colega para ajudar.

Dividir o trabalho

Você tem 100 provas para corrigir e um colega para ajudar.

Cada um corrige 50.

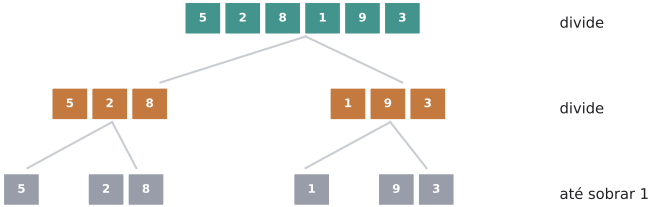
Dividir o trabalho

Você tem 100 provas para corrigir e um colega para ajudar.

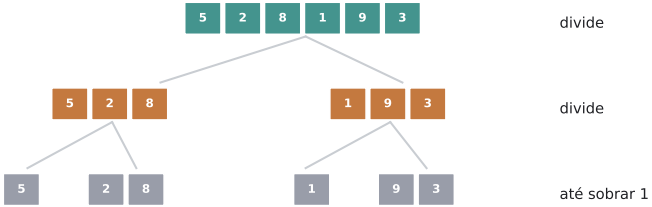
Cada um corrige 50.

É a mesma ideia aqui: divide a lista ao meio,
ordena cada metade, e junta.

Dividindo até não dar mais

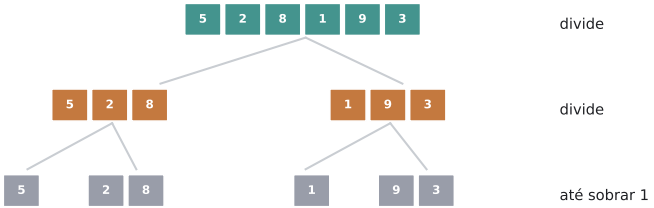


Dividindo até não dar mais



Uma lista de 1 elemento já está ordenada. Nada a fazer.

Dividindo até não dar mais



Uma lista de 1 elemento já está ordenada. Nada a fazer.

O trabalho todo está em *juntar* de volta.

Juntar duas listas ordenadas é fácil

Olhe o primeiro de cada uma. O menor sai.
Repita.

Juntar duas listas ordenadas é fácil

Olhe o primeiro de cada uma. O menor sai.
Repita.

Nunca é preciso voltar atrás.

Juntar duas listas ordenadas é fácil

Olhe o primeiro de cada uma. O menor sai.
Repita.

Nunca é preciso voltar atrás.

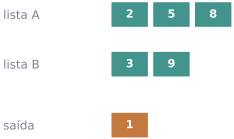
E é isso que torna o método rápido.

Vendo acontecer



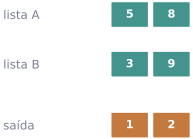
compara 2 e 1

Vendo acontecer



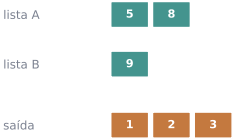
o 1 era menor, saiu. compara 2 e 3

Vendo acontecer



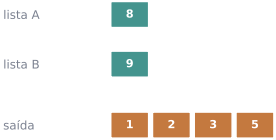
compara 5 e 3

Vendo acontecer



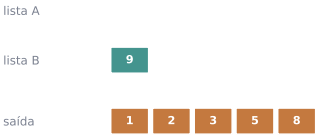
compara 5 e 9

Vendo acontecer



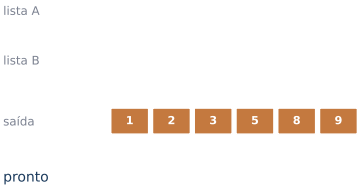
compara 8 e 9

Vendo acontecer

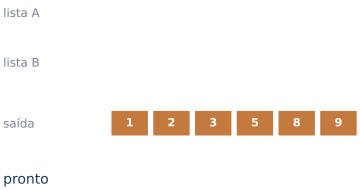


a primeira acabou: o resto vai direto

Vendo acontecer



Vendo acontecer



Cada elemento foi olhado uma vez só.

Por que dá $n \log n$

quantas vezes dá para dividir ao meio?	$\log n$
quanto custa juntar um nível inteiro?	n

Por que dá $n \log n$

quantas vezes dá para dividir ao meio? $\log n$

quanto custa juntar um nível inteiro? n

$\log n$ níveis, cada um custando n.

Por que dá $n \log n$

quantas vezes dá para dividir ao meio?	$\log n$
quanto custa juntar um nível inteiro?	n

$\log n$ níveis, cada um custando n.

Com 2000 números: medi **19.379** comparações.
A inserção fazia **988.698**.

Mas tem um preço

Para juntar duas metades, é preciso uma lista nova.

Mas tem um preço

Para juntar duas metades, é preciso uma lista nova.

A intercalação usa memória extra do tamanho da lista.

Mas tem um preço

Para juntar duas metades, é preciso uma lista nova.

A intercalação usa memória extra do tamanho da lista.

Os métodos da semana passada ordenavam no local, sem gastar nada.

2

A segunda ideia

E se não precisasse juntar?

Na intercalação, a divisão é burra: corta no meio e pronto.
O trabalho fica todo na hora de juntar.

E se não precisasse juntar?

Na intercalação, a divisão é burra: corta no meio e pronto.
O trabalho fica todo na hora de juntar.

E se a divisão já colocasse cada um do lado certo?

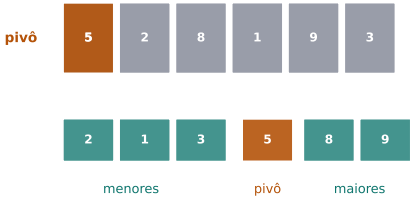
E se não precisasse juntar?

Na intercalação, a divisão é burra: corta no meio e pronto.
O trabalho fica todo na hora de juntar.

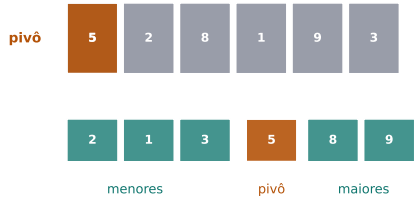
E se a divisão já colocasse cada um do lado certo?

Aí não haveria nada para juntar.

Escolha um elemento e separe

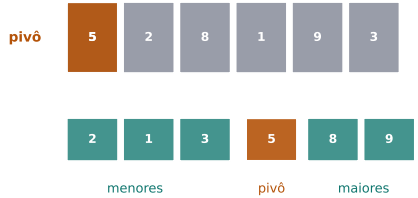


Escolha um elemento e separe



Esse elemento escolhido chama-se **pivô**.
Todos os menores vão para a esquerda, os maiores para a direita.

Escolha um elemento e separe



Esse elemento escolhido chama-se **pivô**.
Todos os menores vão para a esquerda, os maiores para a direita.

O pivô já está no lugar definitivo dele.

E agora repete dos dois lados

Aplique a mesma ideia na parte da esquerda e na da direita.

E agora repete dos dois lados

Aplique a mesma ideia na parte da esquerda e na da direita.

Quando as partes ficarem com 1 elemento, acabou.

E agora repete dos dois lados

Aplique a mesma ideia na parte da esquerda e na da direita.

Quando as partes ficarem com 1 elemento, acabou.

Não há etapa de juntar. Por isso o nome: ordenação *rápida*.

3

O detalhe que estraga tudo

Qual elemento escolher como pivô?

O mais fácil é pegar o primeiro da lista.

Qual elemento escolher como pivô?

O mais fácil é pegar o primeiro da lista.

Funciona. Quase sempre.

Qual elemento escolher como pivô?

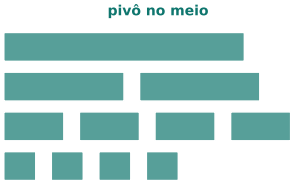
O mais fácil é pegar o primeiro da lista.

Funciona. Quase sempre.

Vamos ver quando não funciona.

O pivô decide o tamanho do corte





O pivô decide o tamanho do corte



Se o pivô for sempre o menor de todos, nada vai para a esquerda.
A lista encolhe de um em um.

O pivô decide o tamanho do corte



Se o pivô for sempre o menor de todos, nada vai para a esquerda.
A lista encolhe de um em um.

E aí são n níveis em vez de $\log n$.

Quando isso acontece

O pivô é o primeiro da lista. Para ele ser sempre o menor...

Quando isso acontece

O pivô é o primeiro da lista. Para ele ser sempre o menor...

...basta a lista já estar ordenada.

Quando isso acontece

O pivô é o primeiro da lista. Para ele ser sempre o menor...

...basta a lista já estar ordenada.

O caso mais comum do mundo real é o pior caso do algoritmo.

Medido

entrada	intercalação	rápida, pivô = primeiro
aleatório	**19.379**	**22.052**
já ordenado	**10.864**	**1.999.000**

Medido

entrada	intercalação	rápida, pivô = primeiro
aleatório	**19.379**	**22.052**
já ordenado	**10.864**	**1.999.000**

Noventa vezes pior, com a lista mais fácil que existe.

Medido

entrada	intercalação	rápida, pivô = primeiro
aleatório	**19.379**	**22.052**
já ordenado	**10.864**	**1.999.000**

Noventa vezes pior, com a lista mais fácil que existe.

E na semana passada, essa mesma lista era o *melhor* caso da inserção.

O conserto é simples

pivô aleatório	sorteie qual elemento usar
mediano de três	olhe o primeiro, o do meio e o último, e use o do meio dos três

O conserto é simples

pivô aleatório	sorteie qual elemento usar
mediano de três	olhe o primeiro, o do meio e o último, e use o do meio dos três

Com pivô aleatório, a mesma lista ordenada deu **25.281** comparações em vez de **1.999.000**.

O conserto é simples

pivô aleatório	sorteie qual elemento usar
mediano de três	olhe o primeiro, o do meio e o último, e use o do meio dos três

Com pivô aleatório, a mesma lista ordenada deu **25.281** comparações em vez de **1.999.000**.

O pior caso continua existindo, mas ficou improvável.

As duas, lado a lado

	intercalação	**rápida**
caso comum	$n \log n$	$n \log n$
pior caso	$n \log n$ sempre	n^2
memória extra	a lista inteira	quase nada
estável	sim	não

As duas, lado a lado

	intercalação	**rápida**
caso comum	$n \log n$	$n \log n$
pior caso	$n \log n$ sempre	n^2
memória extra	a lista inteira	quase nada
estável	sim	não

Uma é previsível. A outra é econômica.

O que as bibliotecas fazem

Nenhuma usa um método puro.

O que as bibliotecas fazem

Nenhuma usa um método puro.

pedaços pequenos	inserção, que é rápida com pouca coisa
pedaços grandes	intercalação ou rápida
trechos já ordenados	detectados e aproveitados

O que as bibliotecas fazem

Nenhuma usa um método puro.

pedaços pequenos	inserção, que é rápida com pouca coisa
pedaços grandes	intercalação ou rápida
trechos já ordenados	detectados e aproveitados

O `sorted` do Python faz exatamente isso.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.
- O preço é a memória extra do tamanho da lista.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.
- O preço é a memória extra do tamanho da lista.
- Rápida: escolhe um pivô e separa menores de maiores.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.
- O preço é a memória extra do tamanho da lista.
- Rápida: escolhe um pivô e separa menores de maiores.
- Com pivô mal escolhido vira $O(n^2)$, e lista ordenada é o pior caso.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.
- O preço é a memória extra do tamanho da lista.
- Rápida: escolhe um pivô e separa menores de maiores.
- Com pivô mal escolhido vira $O(n^2)$, e lista ordenada é o pior caso.
- Pivô aleatório resolve: **25.281** em vez de **1.999.000**.

O que fica

- Intercalação: divide ao meio, ordena cada lado, junta.
- Juntar duas listas ordenadas custa n, e há $\log n$ níveis.
- O preço é a memória extra do tamanho da lista.
- Rápida: escolhe um pivô e separa menores de maiores.
- Com pivô mal escolhido vira $O(n^2)$, e lista ordenada é o pior caso.
- Pivô aleatório resolve: **25.281** em vez de **1.999.000**.

Próximo encontro: como escapar do limite de $n \log n$.